
计算机图形学原理

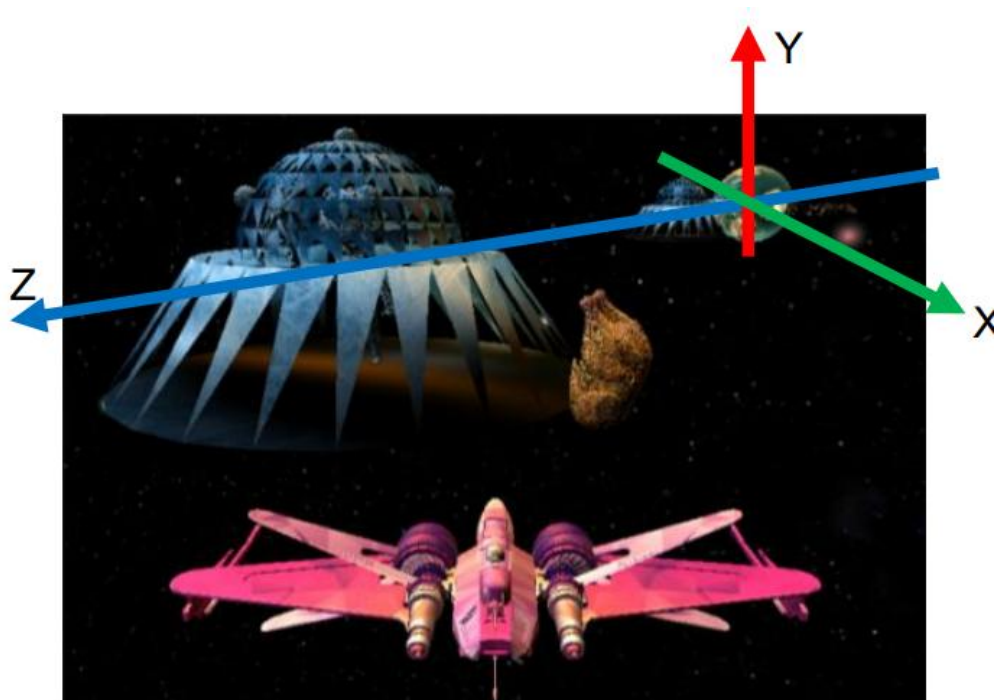
-----教程 11

李瑞辉

5 月教程

- 多个纹理/着色器
- 天空盒
- 法线贴图
- 实例渲染
- 鼠标/键盘交互
- 视点设置
- 视觉反馈
- 碰撞检测

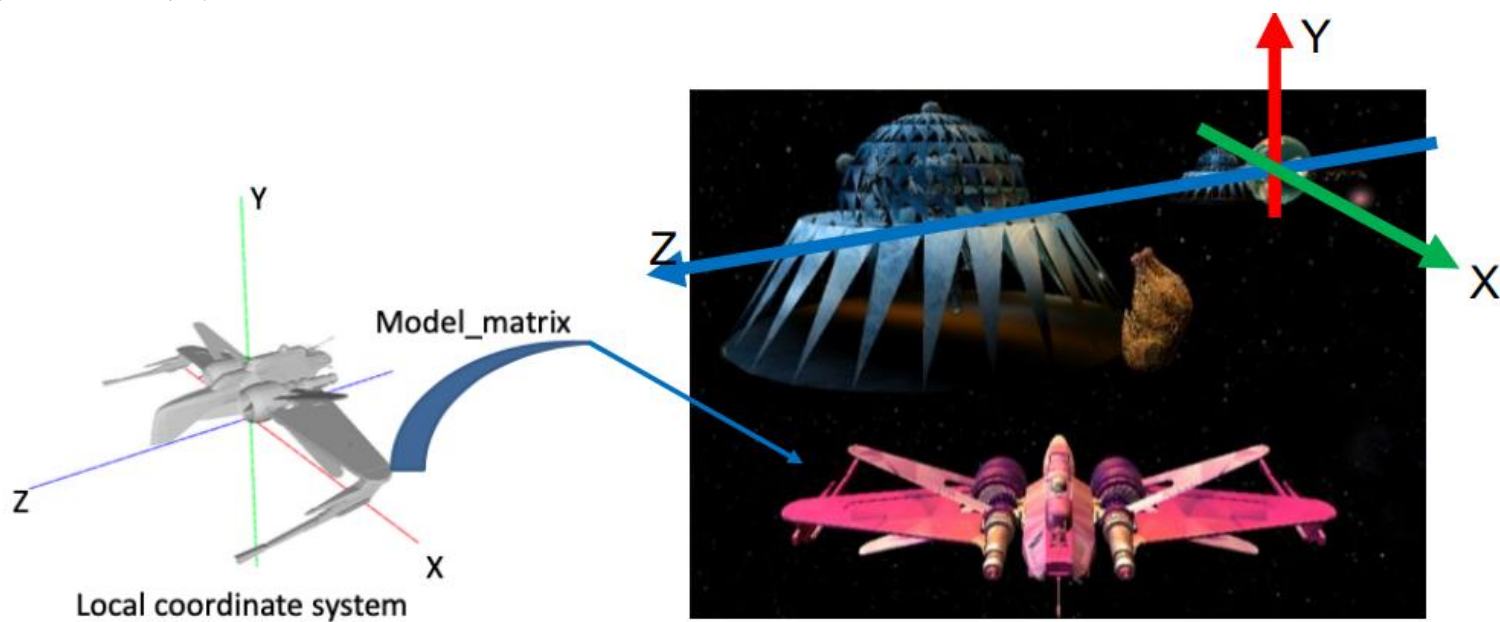
推荐布局



1. 将所有模型放在XYZ平面上。
2. 将行星和外星人沿 Z 轴放置。
3. 使用鼠标和键盘在XYZ平面上自由移动 飞船。

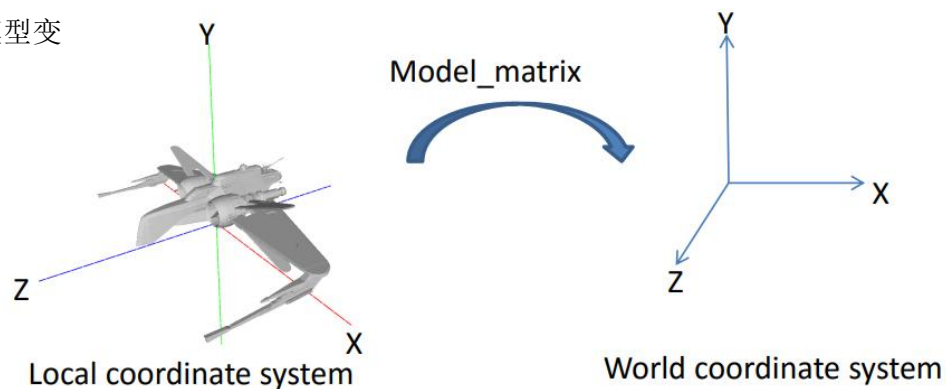
坐标系

1. 航天器局部坐标系。
2. 世界坐标系。
3. 原点在行星的中心。



计算模型矩阵

- 模型是用大尺度定义的，您可以使用缩放矩阵使模型变小
- 旋转和平移
- 如何获得？从鼠标和键盘输入



```
Void UpdateStatus(){  
float scale = 0.0005;  
glm::mat4 SC_scale_M = glm::scale(glm::mat4(1.0f), glm::vec3(scale));  
glm::mat4 SC_trans_M = glm::translate  
(  
glm::mat4(1.0f),  
glm::vec3(SCInitialPos[0] + SCTranslation[0], SCInitialPos[1] + SCTranslation[1], SCInitialPos[2] + SCTranslation[2])  
);  
Model_matrix = SC_trans_M*SC_Rot_M*SC_scale_M;  
}
```

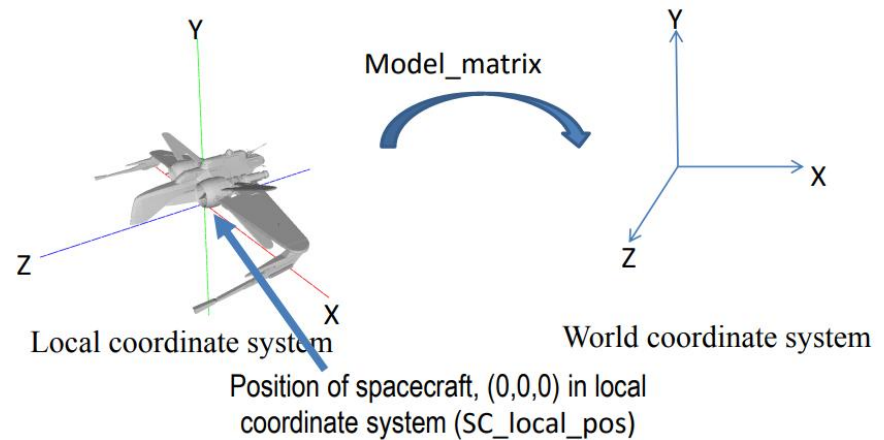
Translation

Rotation

Scale

计算模型矩阵

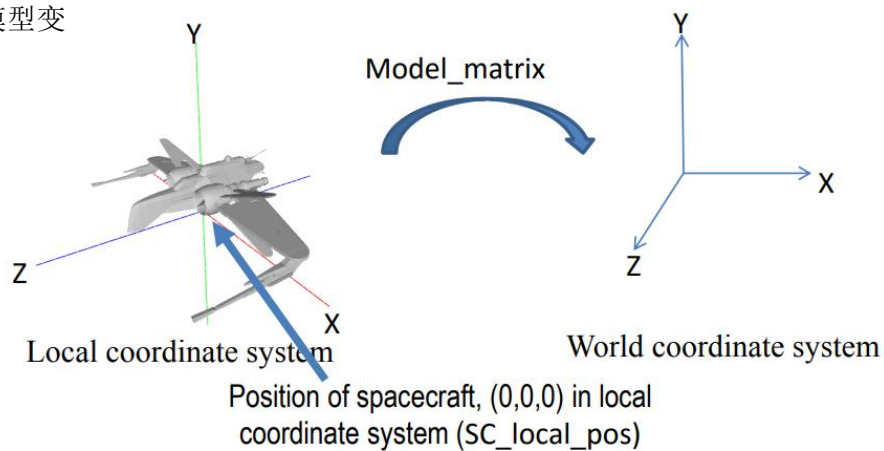
- 模型是用大尺度定义的，您可以使用缩放矩阵使模型变小
- 旋转和平移
- 如何获得？从鼠标和键盘输入



```
Void UpdateStatus(){  
    float scale = 0.0005;  
    glm::mat4 SC_scale_M = glm::scale(glm::mat4(1.0f), glm::vec3(scale));  
    glm::mat4 SC_trans_M = glm::translate  
    (  
        glm::mat4(1.0f),  
        glm::vec3(SCInitialPos[0] + SCTranslation[0], SCInitialPos[1] + SCTranslation[1], SCInitialPos[2] + SCTranslation[2])  
    );  
    Model_matrix = SC_trans_M*SC_Rot_M*SC_scale_M;  
    SC_world_pos = Model_matrix * glm::vec4(SC_local_pos, 1.0f);  
}
```

计算模型矩阵

- 模型是用大尺度定义的，您可以使用缩放矩阵使模型变小
- 旋转和平移
- 如何获得？从鼠标和键盘输入



```
Void UpdateStatus(){  
    float scale = 0.0005;  
    glm::mat4 SC_scale_M = glm::scale(glm::mat4(1.0f), glm::vec3(scale));  
    glm::mat4 SC_trans_M = glm::translate  
    (  
        glm::mat4(1.0f),  
        glm::vec3(SCInitialPos[0] + SCTranslation[0], SCInitialPos[1] + SCTranslation[1], SCInitialPos[2] + SCTranslation[2])  
    );  
    Model_matrix = SC_trans_M * SC_Rot_M * SC_scale_M;  
    SC_world_pos = Model_matrix * glm::vec4(SC_local_pos, 1.0f);  
}
```

通过键盘控制旋转

- Translate along the front and right directions of spacecraft. Get unit vectors of the directions in world space

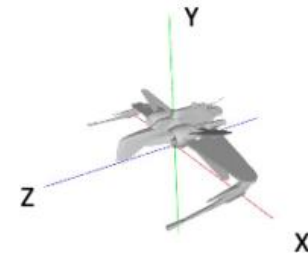
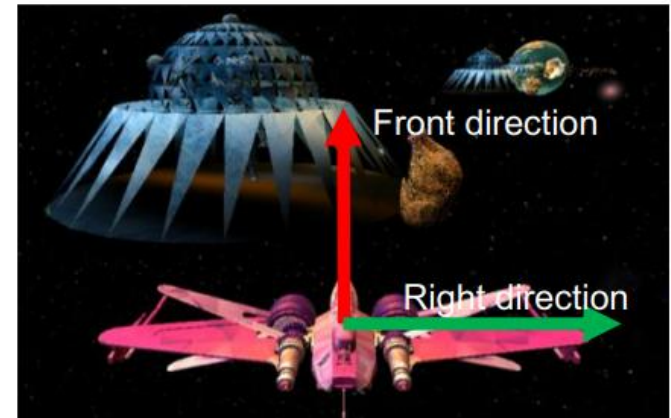
```
void UpdateStatus(){
    float scale = 0.0005;
    glm::mat4 SC_scale_M = glm::scale(glm::mat4(1.0f), glm::vec3(scale));
    glm::mat4 SC_trans_M = glm::translate
    (
        glm::mat4(1.0f),
        glm::vec3(SCInitialPos[0] + SCTranslation[0],
        SCInitialPos[1] + SCTranslation[1], SCInitialPos[2] + SCTranslation[2])
    );
};
```

Model matrix = SC trans M*SC Rot M*SC scale M:

```
SC_world_pos = Model_matrix * glm::vec4(SC_local_pos, 1.0f);
SC_world_Front_Direction = Model_matrix * glm::vec4(SC_local_front, 1.0f);
SC_world_Right_Direction = Model_matrix * glm::vec4(SC_local_right, 1.0f);
SC_world_Front_Direction = Normalize(SC_world_Front_Direction);
SC_world_Right_Direction = Normalize(SC_world_Right_Direction);
};
```

- In main.cpp, register a callback function for keyboard cursor keys

```
int main(int argc, char *argv[])
{
    ...
    glutSpecialFunc(SpecialKeys);
    ...
}
```



SC_local_front = (0,0,-1)

SC_local_right = (1,0,0)

通过键盘控制旋转

- 定义回调函数以获取键盘事件并更新

```
void SpecialKeys(int key, int x, int y)
{
    if (key == GLUT_KEY_DOWN)
    {
        SCTranslation[0] = SCTranslation[0] + 0.5* SC_world_Front_Direction[0];
        SCTranslation[2] = SCTranslation[2] + 0.5* SC_world_Front_Direction[2];
    }
    if (key == GLUT_KEY_UP)
    {
        SCTranslation[0] = SCTranslation[0] - 0.5* SC_world_Front_Direction[0];
        SCTranslation[2] = SCTranslation[2] - 0.5* SC_world_Front_Direction[2];
    }
    if (key == GLUT_KEY_LEFT)
    {
        SCTranslation[0] = SCTranslation [0] - 0.5* SC_world_Right_Direction[0];
        SCTranslation[2] = SCTranslation [2] - 0.5* SC_world_Right_Direction[2];
    }
    if (key == GLUT_KEY_RIGHT)
    {
        SCTranslation[0] = SCTranslation [0] + 0.5* SC_world_Right_Direction[0];
        SCTranslation[2] = SCTranslation [2] + 0.5* SC_world_Right_Direction[2];
    }
}
```

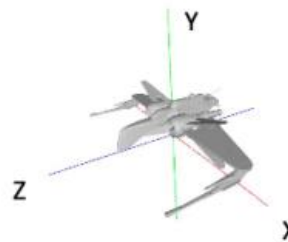
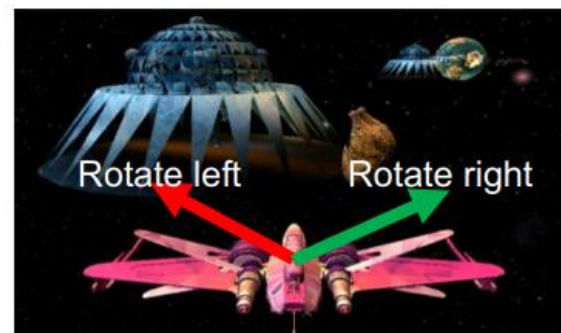
通过鼠标控制旋转

- In main.cpp, register a callback function for mouse movement

```
int main(int argc, char *argv[])
{
    ...
    glutPassiveMotionFunc(PassiveMouse);
    ...
}
```

- Rotate around Y axis in local coordinate system

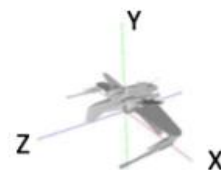
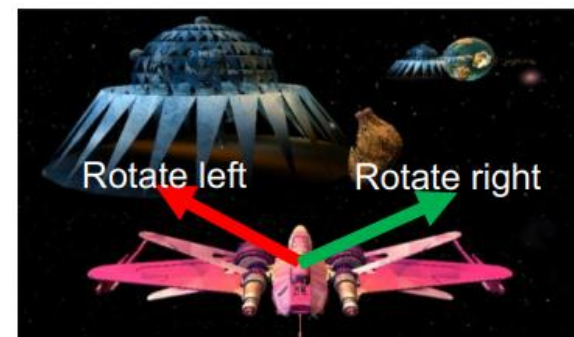
```
void UpdateStatus(){
    float scale = 0.0005;
    glm::mat4 SC_scale_M = glm::scale(glm::mat4(1.0f), glm::vec3(scale));
    glm::mat4 SC_trans_M = glm::translate
    (
        glm::mat4(1.0f),
        glm::vec3(SCInitialPos[0] + SCTranslation[0], SCInitialPos[1] + SCTranslation[1], SCInitialPos[2] + SCTranslation[2])
    );
    Model_matrix = SC_trans_M * SC_Rot_M * SC_scale_M;
    SC_world_pos = Model_matrix * glm::vec4(SC_local_pos, 1.0f);
    SC_world_Front_Direction = Model_matrix * glm::vec4(SC_local_front, 1.0f);
    SC_world_Right_Direction = Model_matrix * glm::vec4(SC_local_right, 1.0f);
    SC_world_Front_Direction = Normalize(SC_world_Front_Direction);
    SC_world_Right_Direction = Normalize(SC_world_Right_Direction);
}
```



通过鼠标控制旋转

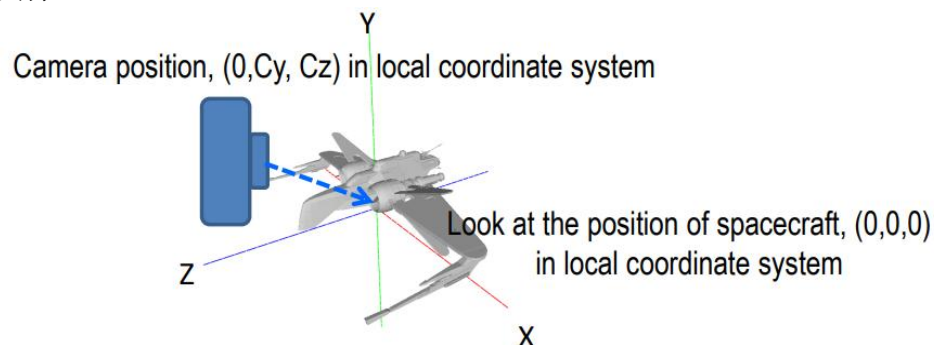
定义鼠标回调函数

```
void PassiveMouse(int x, int y)
{
    if (x<oldx)
    {
        viewRotateDegree[1] += 1.0f;
        SC_Rot_M = glm::rotate(glm::mat4(1.0f), glm::radians(viewRotateDegree[1]), glm::vec3(0.0f, 1.0f, 0.0f));
    }
    if (x>oldx)
    {
        viewRotateDegree[1] -= 1.0f;
        SC_Rot_M = glm::rotate(glm::mat4(1.0f), glm::radians(viewRotateDegree[1]), glm::vec3(0.0f, 1.0f, 0.0f));
    }
    oldx = x;
}
```



视点设定

1. 将虚拟相机固定在航天器上相对静止。
2. 自行设置Cz、Cy

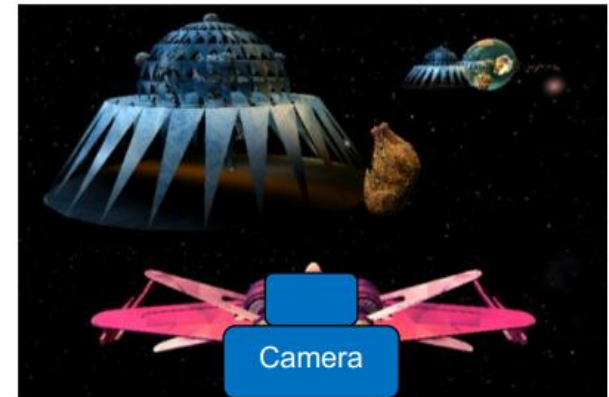
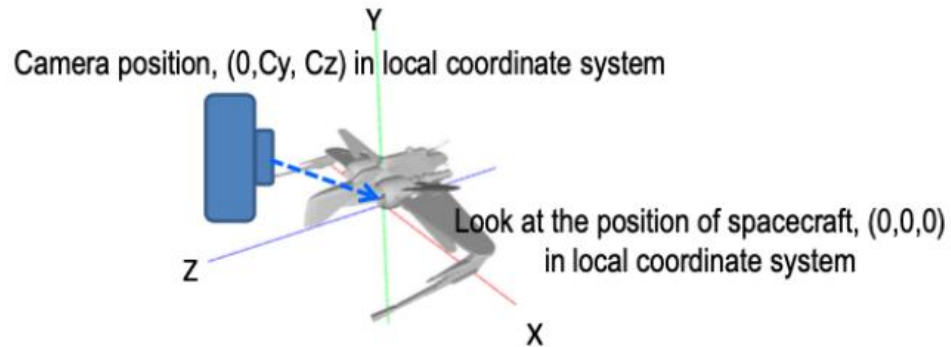


```
void UpdateStatus(){
float scale = 0.0005;
glm::mat4 SC_scale_M = glm::scale(glm::mat4(1.0f), glm::vec3(scale));
glm::mat4 SC_trans_M = glm::translate
(
glm::mat4(1.0f),
glm::vec3(SCInitialPos[0] + SCTranslation[0], SCInitialPos[1] + SCTranslation[1], SCInitialPos[2] + SCTranslation[2])
);
Model_matrix = SC_trans_M*SC_Rot_M*SC_scale_M;
SC_world_pos = Model_matrix * glm::vec4(SC_local_pos, 1.0f);
SC_world_Front_Direction = Model_matrix * glm::vec4(SC_local_front, 1.0f);
SC_world_Right_Direction = Model_matrix * glm::vec4(SC_local_right, 1.0f);
SC_world_Front_Direction = Normalize(SC_world_Front_Direction);
SC_world_Right_Direction = Normalize(SC_world_Right_Direction);
Camera_world_position = Model_matrix * glm::vec4(Camera_local_position, 1.0f);
}
```

视点设定

计算视图矩阵

```
ViewMatrix = glm::lookAt(glm::vec3(Camera_world_position), glm::vec3(SC_world_pos), glm::vec3(0.0f, 1.0f, 0.0f));
```



设置透视矩阵

```
void paintGL(void)
{
    glClearColor(0.0f, 0.0f, 0.8f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);

    // spacecraft modelling
    glm::mat4 translateMatrix = glm::translate(mat4(), vec3(2 + x_translate, 0.5, 15 +
        z_translate));
    glm::mat4 rotateMatrix = glm::rotate(mat4(), glm::radians(y_rotate), vec3(0, 1, 0));
    glm::mat4 scaleMatrix = glm::scale(mat4(), vec3(0.0005f, 0.0005f, 0.0005f));
    glm::mat4 spacecraftModel = translateMatrix * rotateMatrix;
    // world space modelling
    glm::vec4 camera = spacecraftModel * vec4(0.0f, 0.5f, 0.8f, 1.0f);
    glm::vec4 viewport = spacecraftModel * vec4(0.0f, 0.0f, -0.8f, 1.0f);
    // Projection matrix
    glm::mat4 projection = glm::perspective(glm::radians(45.0f), 4.0f / 3.0f, 0.1f, 500.0f);
    glm::mat4 view = glm::lookAt(glm::vec3(camera), glm::vec3(viewport), glm::vec3(0, 1, 0));
    // Model matrix
    glm::mat4 model;
    spacecraftModel = spacecraftModel * glm::rotate(mat4(), glm::radians(180.0f), vec3(0, 1, 0))
        * scaleMatrix;
```

后续教程

- 多个纹理/着色器
- 天空盒
- 法线贴图
- 实例渲染
- 鼠标/键盘交互
- 视点设置
- 视觉反馈
- 碰撞检测

谢谢

